

CONDIÇÕES DE CORRIDA EM TRANSFERÊNCIAS BANCÁRIAS CONCORRENTES

*Análise comparativa de seis mecanismos de sincronização entre threads POSIX e
transações PostgreSQL*

Este relatório descreve o desenvolvimento de um programa em C, com a biblioteca POSIX Pthreads, que simula transferências bancárias concorrentes para evidenciar condições de corrida sobre memória compartilhada e comparar seis estratégias de sincronização quanto à corretude e ao desempenho.

Aluno: Gustavo Vieira de Araujo

Matrícula Aluno: 211068440

Professor: Eduardo A. P. Alchieri

Brasília, 20 de junho de 2026

1. INTRODUÇÃO

Em programas concorrentes, múltiplas threads compartilham o mesmo espaço de memória e se comunicam por variáveis compartilhadas. Quando esse acesso não é coordenado por mecanismos de sincronização (locks, variáveis condição, semáforos), o resultado depende da ordem de intercalamento das instruções pelo escalonador do SO, criando condições de corrida.

O problema escolhido é o de transferências bancárias concorrentes: 10 contas compartilhadas recebem débitos e créditos disparados por várias threads ao mesmo tempo, uma representação do problema de lost update que generaliza o contador compartilhado, pois cada operação manipula duas variáveis (origem e destino), exigindo ordenação de locks para evitar deadlock.

Este trabalho implementa e compara seis estratégias de sincronização: sem controle; “pthread_mutex_t”; “sem_t”; “SELECT FOR UPDATE”; “SERIALIZABLE + retry” e “UPDATE” atômico. Todas compartilham um “motor” produtor-consumidor com fila circular protegida por mutex e variáveis condição.

2. FORMALIZAÇÃO DO PROBLEMA PROPOSTO

2.1 Memória Compartilhada e Invariante

O programa simula um banco com 10 contas (R\$1.000,00 cada), armazenadas em uma tabela PostgreSQL acessada por todas as threads (a memória compartilhada). O total começa em R\$10.000,00 e, como as transferências apenas movem dinheiro, deve permanecer inalterado. Qualquer diferença ao final indica corrupção por condição de corrida.

2.2 Operação de Transferência

Cada transferência move X reais da origem para o destino em quatro passos: T1 (lê saldo da origem), T2 (lê saldo do destino), T3 (grava origem - X), T4 (grava destino + X). Se executados atomicamente, o total não muda. O desafio é garantir essa atomicidade com K threads concorrentes retirando tarefas de uma fila compartilhada (produtor-consumidor).

2.3 Condição de Corrida

A corrida aparece quando duas threads operam sobre a mesma conta: ambas leem o saldo (T1) antes que qualquer uma grave (T3), e a segunda escrita sobrescreve a primeira. A Tabela 1 exemplifica: “A” debita R\$300 e “B” debita R\$400 de uma conta com R\$1.000.

Tabela 1 — Lost update: ambas leem R\$ 1.000 antes de gravar.

Instante	Op. A (-R\$ 300)	Op. B (-R\$ 400)	Saldo
t ₁	Lê → R\$ 1.000	—	R\$ 1.000
t ₂	—	Lê → R\$ 1.000	R\$ 1.000
t ₃	Grava R\$ 700	—	R\$ 700
t ₄	—	Grava R\$ 600 ✗	R\$ 600

O correto seria R\$300. A escrita de B ignorou o débito de A, ou seja R\$300 recriados do nada, violando a invariante.

2.4 Requisitos da Solução

A solução deve garantir, principalmente, a corretude (saldo total preservado para qualquer intercalamento), paralelismo (contas diferentes não devem bloquear umas às outras), bem como ausência de deadlock (sem espera circular ao bloquear duas contas).

3. DESCRIÇÃO DO ALGORITMO DESENVOLVIDO PARA SOLUÇÃO DO PROBLEMA PROPOSTO

3.1 Arquitetura Produtor-Consumidor

Todos os cenários usam a mesma estrutura: um produtor gera 500 transferências aleatórias e insere numa fila circular e depois K workers as consomem, cada um com sua PGconn. A fila é protegida por mutex + duas variáveis condição (not_empty, not_full). Cada worker usa conexão própria para gerar as condições de corrida.

3.2 Cenário 1 — Sem Controle

Executa T1–T4 sem sincronização, com “sched_yield()” entre leitura e escrita para ampliar a janela de corrida e garantir que a corrupção de dados seja observável:

```
double bf = pg_read_balance(c, from); // T1
double bt = pg_read_balance(c, to);   // T2
sched_yield();                         // amplia janela
pg_write_balance(c, from, bf - amt);   // T3
pg_write_balance(c, to, bt + amt);     // T4
```

3.3 Mutex e Semáforo (Cenários 2 e 3)

Tornam T1–T4 atômicos com um lock global (“pthread_mutex_t” ou “sem_t binário”). Garante corretude, mas serializa tudo, ou seja, mesmo contas independentes esperam:

```
pthread_mutex_lock(&g_lock); // ou: sem_wait(&g_sem)
/* T1, T2, T3, T4 */
pthread_mutex_unlock(&g_lock); // ou: sem_post(&g_sem)
```

3.4 Controle no Banco de Dados (Cenários 4, 5, 6)

Os três cenários restantes usam o controle transacional do PostgreSQL. Nos cenários 4 e 6, a ordenação por id evita deadlock.

3.4.1 SELECT FOR UPDATE (Cenário 4)

Abre transação, bloqueia as duas linhas com “SELECT FOR UPDATE” ordenado por id, lê saldos, atualiza e faz “COMMIT”. Bloqueios por linha permitem paralelismo entre contas diferentes:

```
int a = from < to ? from : to; // menor id primeiro
pg_exec_q(c, "BEGIN;");
"SELECT ... WHERE id IN(a,b) ORDER BY id FOR UPDATE;"
/* lê saldos, atualiza, COMMIT */
```

3.4.2 SERIALIZABLE + Retry (Cenário 5)

Abordagem otimista: executa com isolamento “SERIALIZABLE”; se o banco detectar conflito (SQLSTATE 40001), a aplicação refaz a transação do zero. O contador “g_retries” mede a contenção:

```
for (;;) {
    pg_exec_q(c, "BEGIN ISOLATION LEVEL SERIALIZABLE;");
    /* T1, T2, T3, T4 */
    if (pg_exec_q(c, "COMMIT;")) break; // sucesso
    g_retries++;                       // refaz tudo
}
```

3.4.3 UPDATE Atômico (Cenário 6)

Elimina o ler-calcular-gravar: o banco faz tudo com “UPDATE balance = balance + X”. Nenhum valor sai para a memória da thread, eliminando a janela de corrida. Apenas 2 “UPDATES” por transferência (melhor que as 4 chamadas nos outros cenários):

```
int first = from < to ? from : to; // ordenação por id
pg_exec_q(c, "BEGIN;");
"UPDATE accounts SET balance=balance+X WHERE id=first;"
"UPDATE accounts SET balance=balance+X WHERE id=second;"
pg_exec_q(c, "COMMIT;");
```

4. CONCLUSÃO

O cenário 1 confirma o problema: o saldo total diverge de forma diferente toda vez que são executados os testes. Nos cenários 2–6, o saldo é preservado em todas as execuções. Mutex e semáforo (2–3) usam lock global (corretos, porém sem paralelismo). O “SELECT FOR UPDATE” permite paralelismo entre contas diferentes, com mais queries. O SERIALIZABLE não bloqueia, mas refaz transações sob contenção. O UPDATE atômico obteve o melhor resultado: locks curtos, paralelismo e apenas 2 “UPDATES” por transferência.

A ordenação por id (cenários 4 e 6) foi essencial para evitar deadlock entre threads transferindo em direções opostas no mesmo par de contas (prevenção por ordenação de recursos). A lição central: exclusão mútua pode ser garantida em diferentes camadas (aplicação ou banco), mas a granularidade do bloqueio determina o paralelismo real. O cenário 6 (melhor solução) vai além, visto que em vez de serializar ao redor da janela de corrida, elimina a própria janela movendo a operação para dentro do banco.

5. REFERÊNCIAS

ALCHIERI, E. A. P. Notas de aula — Programação Concorrente. Brasília: UnB, 2026.

THE OPEN GROUP. POSIX.1-2017 — pthread_mutex_lock, pthread_cond_wait, sem_wait. <https://pubs.opengroup.org/onlinepubs/9699919799/>

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 — Concurrency Control. <https://www.postgresql.org/docs/16/mvcc.html>